Question 1 Answer: Coding against an interface promotes flexibility and decoupling, making it easier to swap implementations without changing dependent code.

Question 2 Answer: Use an abstract class when you need shared code or state; use an interface when you only need to define a contract.

Question 3 Answer: Implementing IComparable allows objects to define their own sorting logic, making collections more flexible and reusable.

Question 4 Answer: Explicit interface implementation resolves naming conflicts by separating interface methods from class methods with the same name.

Question 5 Answer: Both provide encapsulation, but structs are value types stored on the stack, while classes are reference types stored on the heap.

Question 6 Answer: Abstraction hides implementation details and shows only essentials, while encapsulation controls access to data; abstraction is the design principle, encapsulation is its enforcement.

Question 7 Answer: Default interface implementations let you add new methods to interfaces without breaking existing implementations, improving backward compatibility.

Question 8 Answer: Constructor overloading improves usability by allowing objects to be initialized in multiple ways depending on available data